

Projectile Motion Simulator

Official Documentation

Version 3.2.0

Educational Physics Simulation Tool

Based on the Technology-Mediated Strategic Intervention Material
developed by Sir Rene D. Mistiola

Department of Education — Schools Division of Batangas

April 2026

Table of Contents

1. Overview
2. User Guide
3. Physics Explanation
4. Technical Documentation
5. Installation & Setup
6. Known Limitations
7. Future Improvements

1. Overview

The Projectile Motion Simulator is an interactive, browser-based educational tool designed to help senior high school students visualize and understand the physics of projectile motion. It transforms the abstract equations found in General Physics 1 into a concrete, animated, and explorable experience.

1.1 Purpose

Built to support the Technology-Mediated Strategic Intervention Material (TMSIM) developed by Sir Rene D. Mistiola, the simulator allows learners to manipulate launch parameters in real time and observe how each variable influences the resulting trajectory, range, time of flight, and maximum height.

1.2 Target Audience

- Grade 11 and Grade 12 General Physics 1 students
- Physics teachers preparing demonstrations or lab activities
- Self-directed learners exploring kinematics

1.3 Key Features

- Real-time animated simulation with 60 FPS canvas rendering
- Adjustable initial velocity, launch angle (degrees or radians), height, mass, and gravity
- Optional linear air drag with adjustable coefficient
- Dual display modes: Educational (rounded) and Physics (precise)
- Live, Final, and Target stat cards — draggable and collapsible
- Mobile and desktop view layouts with persistent preference
- Futuristic sound effects, dismissible onboarding overlay, and dark/light themes
- Preset scenarios for quick exploration

2. User Guide

2.1 Getting Started

On first launch, an onboarding overlay appears with a Quick Start guide. Dismiss it once and it stays hidden on subsequent visits (stored in browser localStorage).

2.2 Control Panel

- Initial Velocity (m/s) — speed at the moment of launch
- Launch Angle — adjustable in degrees or radians
- Initial Height (m) — vertical launch position above ground
- Mass (kg) — used when air drag is enabled
- Gravity (m/s^2) — defaults to 9.8; adjustable to simulate other planets
- Air Drag Toggle + Coefficient — enables linear drag $F = -k \cdot v$

2.3 Playback Controls

- Launch — starts the simulation
- Pause / Resume — toggles animation
- Step — advances one fixed time step while paused
- Reset — returns the projectile to its starting position
- Time Scale — slows or speeds up animation without affecting physics

2.4 Display Options

- Grid overlay, velocity vectors, and trajectory trail toggles
- Target Mode — drop a target marker and measure miss distance
- Educational Mode — rounds values for classroom clarity
- Physics Mode — shows precise floating-point values

2.5 Stats HUD

Three floating cards display Live values (current velocity, position, time), Final values (range, flight time, max height), and Target metrics. Each card can be dragged anywhere on screen and collapsed; positions persist across sessions.

2.6 Mobile vs Desktop View

A toggle in the header switches between a stacked mobile-friendly layout and a side-by-side desktop layout. The choice is remembered.

2.7 About & Documentation

The About dialog (rocket icon in the header) describes the app, credits the proponent, and provides direct download links to this documentation in PDF and DOCX formats.

3. Physics Explanation

3.1 Coordinate System

The simulator uses a standard 2D Cartesian system: the x-axis points horizontally in the launch direction and the y-axis points vertically upward. Gravity acts in the $-y$ direction with magnitude g (default 9.8 m/s^2).

3.2 Initial Velocity Components

$$V_x = V_i \cdot \cos(\theta)$$

$$V_y = V_i \cdot \sin(\theta)$$

Where V_i is the initial speed and θ is the launch angle measured from the horizontal.

3.3 Time of Flight

For a projectile launched from height h above the landing surface:

$$t = (V_y + \sqrt{ V_y^2 + 2 \cdot g \cdot h }) / g$$

3.4 Maximum Height

$$H = V_y^2 / (2 \cdot g) + h$$

3.5 Horizontal Range

$$R = V_x \cdot t$$

3.6 Air Drag Model

When drag is enabled, a linear drag force opposes velocity:

$$F_{\text{drag}} = -k \cdot v$$

This is integrated step-by-step using a semi-implicit Euler scheme. Linear drag is a simplification chosen for stability and pedagogical clarity; it does not capture quadratic (high-Reynolds) air resistance.

3.7 Analytical vs Simulated Results

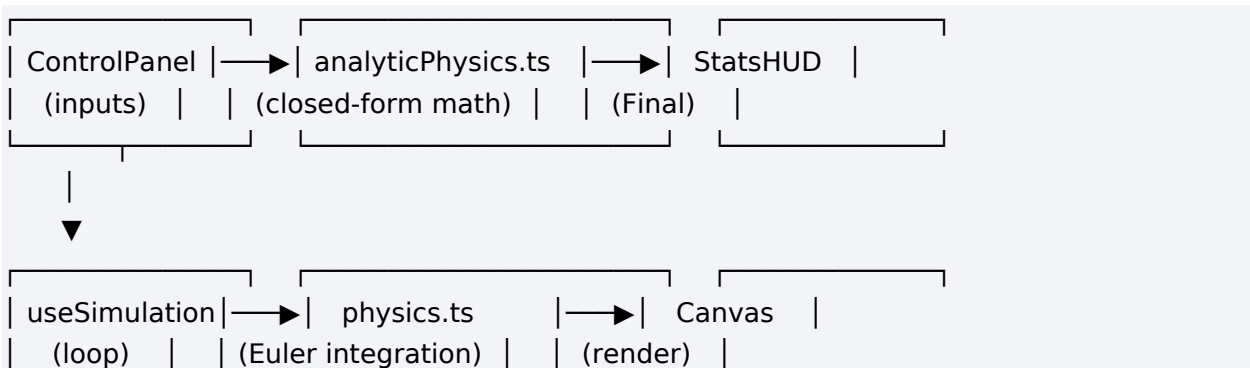
The HUD displays analytical (closed-form) results from src/lib/analyticPhysics.ts as the source of truth. The animated trajectory is generated by the simulation loop in src/lib/physics.ts using fixed time steps. With drag disabled, the two agree to within floating-point precision; with drag enabled, the simulation deviates from the drag-free analytical formulas as expected.

4. Technical Documentation

4.1 Tech Stack

- React 18 with TypeScript 5
- Vite 5 build tool and dev server
- Tailwind CSS v3 with shadcn/ui components (Radix primitives)
- HTML5 Canvas for trajectory rendering
- WebAudio API for futuristic sound effects
- Vitest for unit testing

4.2 Architecture Overview



4.3 Module Breakdown

- `src/lib/analyticPhysics.ts` — exact closed-form formulas; single source of truth for HUD stats
- `src/lib/physics.ts` — semi-implicit Euler integrator for the animated trajectory
- `src/hooks/useSimulation.ts` — React hook orchestrating the `requestAnimationFrame` loop and state
- `src/lib/sounds.ts` — WebAudio-based SFX (clicks, ticks, launch, landing)
- `src/components/SimulatorHeader.tsx` — top bar with view toggle, theme, About
- `src/components/ControlPanel.tsx` — sliders, inputs, presets, playback buttons
- `src/components/SimulationCanvas.tsx` — canvas rendering of trajectory, grid, vectors
- `src/components/StatsHUD.tsx` + `DraggableCard.tsx` — floating draggable stat cards
- `src/components/AboutDialog.tsx` — credits and documentation downloads
- `src/components/OnboardingOverlay.tsx` — first-visit Quick Start

4.4 State Persistence (localStorage)

- View mode preference (mobile vs desktop)
- Onboarding-dismissed flag
- Position and collapse state of each draggable HUD card
- Theme preference (dark/light)

4.5 Testing

Unit tests run with Vitest. The setup file lives at `src/test/setup.ts` and an example test at `src/test/example.test.ts`. Run ``npm run test`` (single pass) or ``npm run test:watch`` for development.

5. Installation & Setup

5.1 Prerequisites

- Node.js 18 or newer
- npm (bundled with Node) or bun
- Modern browser with WebAudio support (Chrome, Edge, Firefox, Safari)

5.2 Local Setup

```
git clone <your-repo-url>
cd projectile-motion-simulator
npm install
npm run dev
```

Then open `http://localhost:8080` in your browser.

5.3 Available Scripts

- `npm run dev` — start Vite dev server
- `npm run build` — production build
- `npm run build:dev` — development-mode build
- `npm run preview` — preview the production build locally
- `npm run lint` — ESLint check
- `npm run test` — run Vitest once
- `npm run test:watch` — Vitest in watch mode

5.4 Deployment

The app is statically built and can be hosted on any static host. The current published build is available at <https://mistiola-projectile-v3-2-beta.lovable.app>.

6. Known Limitations

- Air drag uses a linear model only ($F = -kv$); high-speed quadratic drag is not modeled
- No wind, spin, or Magnus-effect simulation
- Strictly 2D — no 3D trajectories
- Only one projectile is simulated at a time
- Ground is fixed at $y = 0$; no terrain or inclined surfaces
- WebAudio requires a user gesture before sound plays (browser policy); the first interaction unlocks audio
- On very slow mobile devices, the bottom sheet may briefly overlap the canvas during open/close animation

7. Future Improvements

- Quadratic air drag with adjustable air density and cross-sectional area
- Multiple simultaneous projectiles for side-by-side comparison
- Variable terrain and inclined ground
- Energy graphs (kinetic, potential, total) plotted over time
- Export trajectory data to CSV for analysis in spreadsheets
- Built-in problem sets with auto-grading
- Multilingual UI (English and Filipino)
- Progressive Web App (PWA) install for offline classroom use
- Optional 3D mode using WebGL

— End of Documentation —